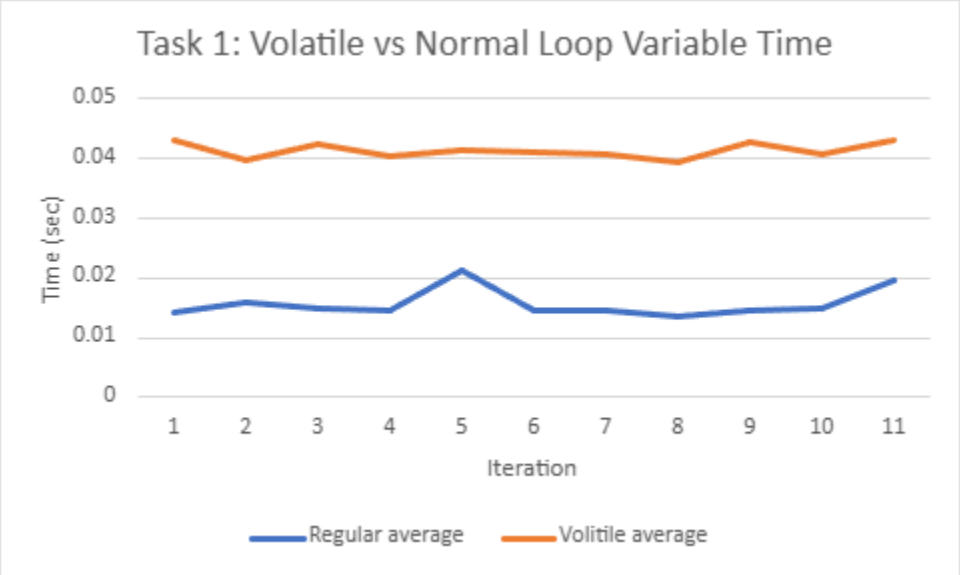
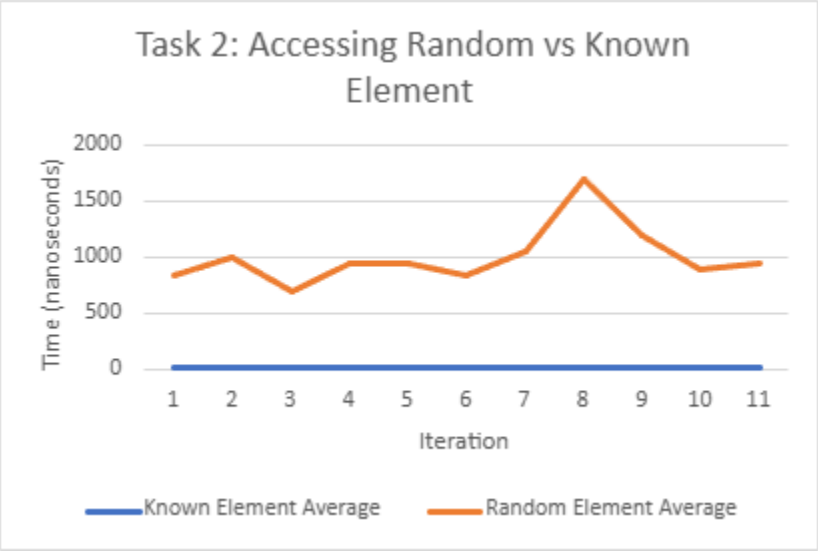
Task 1:

Performing task 1, I noticed that the average time taken to complete the requested task of running the loop variable which was marked as “volatile” always took significantly longer to complete than that of the normally defined loop variable. This is of course because the “volatile” key word informs the compiler that it is not mean to go to a cache for the variable value, rather always going to main memory which takes much longer than simply caching it. In none of the iterations of the tests I performed of task 1 did the “volatile” average time even come close to that of the normal loop variable which goes to demonstrate the consistency of the fact that a variable marked with the keyword “volatile” will always be slower in accessing than that of a normally defined variable due to the fact of how traversing the memory hierarchy works



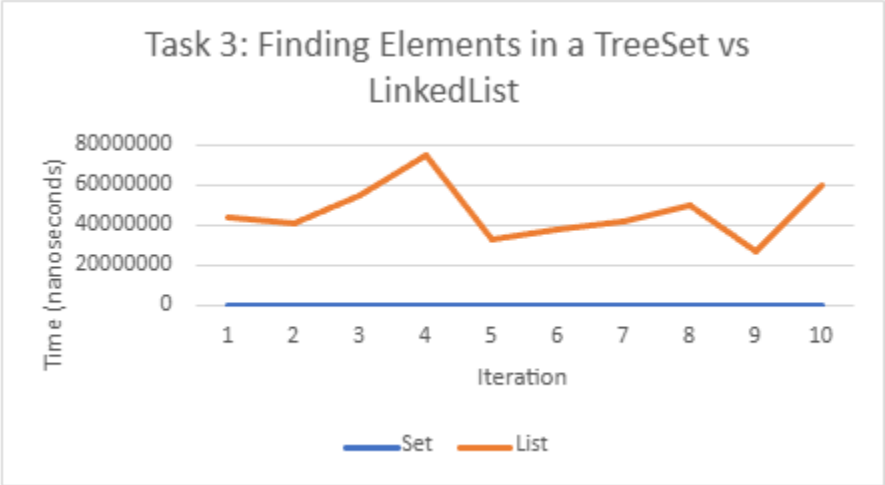
Task 2:

In performing task 2, I noticed that the average time to access a known element from an ArrayList varied massively from that of accessing a random element. So much so that the difference can barely be seen on the graph, while the random element ranged in times from 700 - 1700 nanoseconds, the known element registered in a range of 22-23 nanoseconds. This massive difference in time between the two is due to how caching works by placing the next element in a given sequence next in line in the cache making it very easy to move from one to the next in order predictively allowing for a very short amount of time to elapse from accessing each element while making accessing a random element much much harder since it is completely unknown where the element would be and therefore taking a while to find it. This would also account for the massive variability in the times of the random element because in some cases it would be easier to find due to being closer and others longer due to greater distance.



Task 3

For task 3, I noticed that finding an element in a TreeSet was magnitudes faster than finding that same element in a Linked list. Much like with task 2, my results from task 3 can barely be properly displayed on a graph due to the fact of the average time to find an element in a linked list being scaled in the orders of 10^7 while the corresponding time in a TreeSet is measured in the relatively mere 10^4. This massive time difference, I believe, is due to the structure of the two data structures. Linked lists are very linear and therefore when they are tasked to find a random element which happens to be at the end of said list, it will take a lot longer to find than in a Tree which has the elements split into different branches which are meant for quick traversal. Therefore, the Tree system is designed perfectly for this kind of test while the linked list struggles immensely.



Here is a link to the spreadsheet with the tables and graphs
[Homework 2 Graphs]